
Proctored Interview Module

Implementation Plan

Plan for an AI-proctored mock interview module: admin uploads resume + JD, system generates a question set and a passcode-protected one-time meeting link, and the student takes a fully supervised session with real-time browser-side detection (eyes, hands, facial expression, prohibited objects, multiple faces, background noise, tab/fullscreen) plus real-time English proficiency and fluency measurement, 3-warning auto-termination, and an admin-only analysis report.

Project	Creole Knowledge Portal
Document type	Technical implementation plan
Status	Planning, pending approval
Date	8 September 2026

Contents

1. Assumptions
2. Requirement coverage
3. Library choices
4. How the vision detection loop works

5. English proficiency and fluency
6. User smartness
7. Warning and termination policy
8. Data model
9. Route map
10. Full flow
11. Session integrity details that are easy to miss
12. Report contents for the admin
13. Build order
14. Risks

Proctored Interview Module

Assumptions

These are the defaults this plan is built on. Any of them can be changed before implementation starts.

- "Meeting" is a self-paced automated session (no human interviewer, no WebRTC peer).
 - "User smartness" = Gemini composite: answer correctness/depth/reasoning plus behavioral confidence (response latency, hesitation, filler rate).
 - All detection runs 100% in the student's browser; only small JSON events reach the server. Free, private, real-time.
 - Report lands on an admin dashboard page. The repo has no email provider installed, so email delivery is a flagged add-on in Stage 8.
 - Student is anonymous (no Supabase account). Their writes go through service-role routes gated by the invite token plus session cookie, not `auth.uid()` RLS.
-

Requirement coverage

Every requirement from the brief, mapped to the stage that handles it.

- Resume + JD produce a question set and a meeting link — Stage 2
- Link passcode protected, opens only once — Stage 1 and 3
- Instruction page before start — Stage 3
- Webcam and full screen required — Stage 4
- Eye movement — Stage 5
- Hand movement — Stage 5
- Facial expression — Stage 5
- **English fluency and proficiency** — Stage 6 and 7, detailed below
- User smartness — Stage 7
- No phone, book, or second laptop in frame — Stage 5
- Background noise detection — Stage 6
- Multiple face detection — Stage 5
- Real-time warnings, auto-terminate after 3 — Stage 5
- Tab change terminates the meeting — Stage 4
- Thank-you screen for the student, full report to admin — Stage 8

Also in scope, carried over from earlier discussion:

- The session is a **spoken mock interview only**. Unlocking the link leads straight to the interview, with no session type to choose.
- The student never sees any score. The end screen says only: **"Your test is submitted. We will come back to you in sometime."**

- Question difficulty is driven by the candidate's experience level from the resume versus the JD's seniority.

Explicitly out of scope: written quiz assessments, MCQ grading, and any Quiz or Both session option.

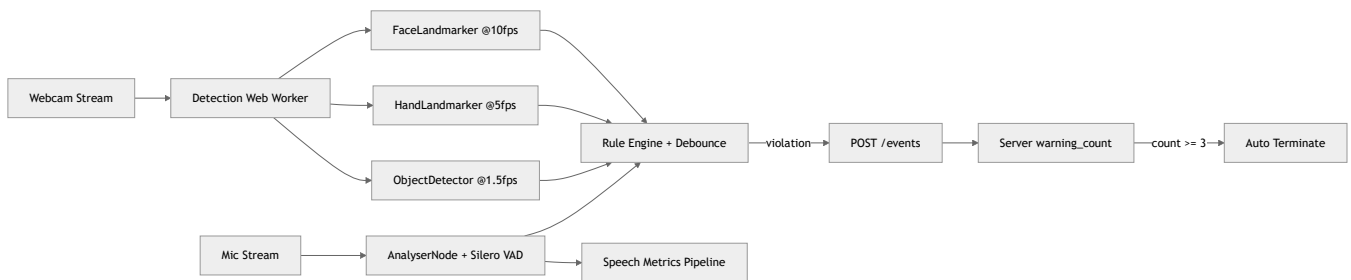
Library choices

Need	Library	Why
PDF resume/JD text	<code>unpdf</code>	Serverless-safe, no native binaries
DOCX text	<code>mammoth</code>	Standard DOCX to text
Question gen, grading, English rubric, smartness	<code>@google/genai</code> (already installed)	Reuse the model-cascade pattern in lib/ai/quiz-generator.ts
Eye movement, facial expression	<code>@mediapipe/tasks-vision</code> FaceLandmarker	478 landmarks + iris + 52 ARKit blendshapes in one model
Hand movement	<code>@mediapipe/tasks-vision</code> HandLandmarker	21 landmarks per hand, up to 2 hands
Multiple faces	Same FaceLandmarker with <code>numFaces: 3</code>	Avoids loading a second face model
Phone / book / laptop	<code>@mediapipe/tasks-vision</code> ObjectDetector + EfficientDet-Lite0	COCO classes include <code>cell phone</code> , <code>book</code> , <code>laptop</code>
Speech vs background noise	<code>@ricky0123/vad-web</code> (Silero VAD) + Web Audio <code>AnalyserNode</code>	VAD separates speech from noise; RMS gives the noise level
Speech to text	Web Speech API (<code>webkitSpeechRecognition</code>)	Free, native, Chromium; Gemini audio as fallback
English fluency metrics	<code>@ricky0123/vad-web</code> timeline + own <code>lib/interview/fluency.ts</code>	Deterministic math (WPM, pauses, fillers), no model needed
English proficiency (grammar, vocabulary, CEFR)	<code>@google/genai</code>	Language quality needs an LLM, not arithmetic
Screen share + webcam	<code>getDisplayMedia</code> / <code>getUserMedia</code>	Extend existing lib/quizzes/proctor.ts
Fullscreen lock	Fullscreen API	<code>requestFullscreen()</code> plus <code>fullscreenchange</code> listener
Tab switch	<code>visibilitychange</code> / <code>blur</code>	Immediate terminate per your requirement
Token and passcode hashing	<code>node:crypto</code> (<code>randomBytes</code> , <code>scrypt</code>)	No new dependency

Only four new packages: `unpdf`, `mammoth`, `@mediapipe/tasks-vision`, `@ricky0123/vad-web`.

How the vision detection loop works

Three models at full frame rate would jank the UI, so they run in a Web Worker with `OffscreenCanvas` at staggered rates.



Per-signal rules, all debounced so a half-second glance never fires:

- Eye movement: iris centre offset inside the eye socket plus head yaw/pitch from landmarks. Away from screen for 2s continuous fires `gaze_away`.
- Facial expression: blendshape vector, watching `jawOpen`, `browDownLeft/Right`, `eyeSquint`, `mouthPress`. Sustained anomaly fires `expression_anomaly` and also feeds the confidence input for smartness.
- Hand movement: hand landmarks entering the face or mouth region, or leaving frame repeatedly, fires `suspicious_hand`.
- Objects: any detection of `cell phone`, `book`, or `laptop` above 0.5 confidence across 3 consecutive inferences fires `prohibited_object` with the class name.
- Multiple faces: `faces.length >= 2` for 1s fires `multi_face`; `0` faces for 3s fires `face_missing`.
- Background noise: RMS above threshold while VAD reports non-speech for 3s fires `background_noise`.
- Tab change, fullscreen exit, screen-share stop, camera stop: immediate, no debounce.

The server owns the counter. Each event POST returns `{ warning_count, terminated }`, so a student cannot cheat by tampering with the client-side counter.

English proficiency and fluency

This is a **scored** signal, not a warning signal. Poor English never terminates the session. The only speech-related warning is prolonged silence, so a student cannot idle out the clock.

Two different things, measured two different ways:

- **Fluency** is delivery mechanics: how fast, how smoothly, how much hesitation. Pure arithmetic from timing plus transcript, no AI, so it is cheap, instant, and reproducible.
- **Proficiency** is language quality: grammar, vocabulary, coherence, CEFR band. Needs an LLM, so it runs on the server after each answer.

Libraries used for English detection

Sub-signal	Library / API	What it gives us
Speech vs silence boundaries	<code>@ricky0123/vad-web</code> (Silero VAD, ONNX in browser)	<code>onSpeechStart</code> / <code>onSpeechEnd</code> timestamps, so we know true speaking time
Words spoken	Web Speech API <code>SpeechRecognition</code>	Interim and final transcripts, plus <code>results[i][0].confidence</code>
Audio level and noise floor	Web Audio <code>AnalyserNode</code>	RMS energy, used for noise events and low-mic-quality flags
WPM, pause ratio, filler rate, repetition	Own module <code>lib/interview/fluency.ts</code>	Deterministic formulas over the VAD timeline plus transcript
Grammar, vocabulary, coherence, CEFR band	<code>@google/genai</code> (Gemini)	JSON rubric per answer
Fallback transcription (non-Chromium)	Gemini audio input	Used when <code>SpeechRecognition</code> is unavailable or confidence is low

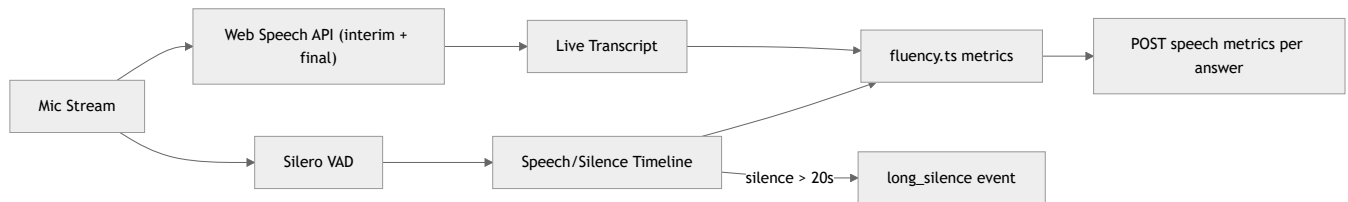
Nothing here is paid, and only `@ricky0123/vad-web` is a new dependency for this signal.

How fluency is detected, step by step

1. Mic stream opens once at setup via `getUserMedia({ audio: true })`.
2. Silero VAD subscribes to that stream and emits speech-start and speech-end events, building a timeline of speech segments and silence gaps for the current answer.
3. In parallel, `SpeechRecognition` runs in continuous mode with interim results, accumulating the transcript and per-result confidence.
4. When the student finishes an answer, `lib/interview/fluency.ts` computes from the two sources:
 - `speech_ms` = sum of VAD speech segments; `silence_ms` = answer duration minus `speech_ms`
 - `wpm = words / (speech_ms / 60000)` — divided by real speaking time, so thinking pauses never inflate the score
 - `pause_ratio = silence_ms / answer_duration_ms`
 - `filler_rate = filler_matches / words`, matched against a lexicon of um, uh, er, like, you know, actually, basically, I mean
 - `repetition_rate` = repeated bigrams and restart markers over total words
 - `speech_segments` = VAD segment count, where many tiny fragments signals hesitancy
5. Those numbers are normalised into a 0 to 100 fluency score using bands: 110 to 160 WPM is ideal and anything below 90 or above 190 loses points, pause ratio under 0.20 is good and over 0.35 is poor, filler rate under 2 percent is good and over 5 percent is poor.
6. The metrics plus the transcript are POSTed with the answer and stored in `interview_speech_metrics`.
7. The server sends the transcript and timings to Gemini for the proficiency rubric, and the final English score combines the deterministic fluency score with the Gemini proficiency score.

Steps 1 through 5 run entirely in the browser in real time. Step 7 is the only part that costs an API call, and it happens once per answer rather than continuously.

Pipeline diagram



Prolonged silence over 20s raises **long_silence**, which prompts the student and counts as a warning only if it repeats.

The student sees only a mic-active indicator and the live transcript. No fluency score is shown, to avoid performance anxiety and to keep all scoring admin-only.

Post-answer layer (server, Gemini)

Each transcript plus its timing metadata goes to Gemini for a language rubric returning strict JSON:

- Grammar accuracy
- Vocabulary range and appropriateness for the JD seniority
- Sentence complexity and coherence
- Discourse markers and structure
- CEFR-style band estimate from A2 to C2
- Short qualitative feedback for the admin

Pronunciation caveat: true phoneme-level pronunciation scoring needs a paid service such as Azure Speech pronunciation assessment. We approximate it with Web Speech API per-result confidence plus Gemini's judgement of intelligibility, and label it clearly as an estimate in the report.

Final English score

The deterministic fluency score and the Gemini proficiency score combine into a single English score for the interview, with both sub-scores kept visible to the admin so a fast talker with weak grammar is not confused with a careful speaker who pauses.

User smartness

A composite, so no single weak signal dominates:

- Answer quality from the Gemini grader: correctness, depth, reasoning quality, use of concrete examples.
- Relevance of answers to the JD's must-have skills.
- Response latency: time from the question being shown to the start of speech, normalised per question difficulty.
- Confidence signals: filler rate, hesitation, self-corrections, plus the facial expression stream.

Reported as a 0 to 100 score with the contributing sub-scores listed, never as a single opaque number.

Warning and termination policy

- Soft violations (gaze, hands, expression, prohibited object, multiple faces, background noise, repeated long silence) increment `warning_count`. The client shows a red banner such as "Warning 2 of 3: phone detected".
 - On the third warning the server sets `end_reason = 'auto_terminated_warnings'`, rejects further answer POSTs, stops the streams, and the thank-you screen appears.
 - Tab change or fullscreen exit terminates immediately with `end_reason = 'tab_switch'`. A config flag `terminate_on_tab_switch` (default true) lets you soften this to a warning after a pilot.
 - Camera or screen-share revoked mid-session gives a 15-second restore grace, then terminates with `end_reason = 'media_revoked'`.
 - Each violation optionally stores a small JPEG snapshot of the webcam frame in a private bucket so the admin can verify the call.
-

Data model

New migration `supabase/migrations/<ts>_create_interview_module.sql`:

- `interview_sessions` — resume and JD storage paths, `parsed_resume`, `parsed_jd`, `skill_gap`, `experience_level`, `created_by`, `status`.
- `interview_invites` — `token_hash`, `passcode_hash` and `passcode_salt`, `status` (active, in_progress, completed, expired, revoked), `expires_at`, `bound_session_id`, `max_warnings` default 3.
- `interview_questions` — the generated bank, `intent` (technical, system design, behavioural, role fit), `difficulty`.
- `interview_answers` — response, `transcript`, `time_to_first_response_ms`, `time_taken_ms`.
- `interview_speech_metrics` — per answer: `wpm`, `pause_ratio`, `filler_rate`, `repetition_rate`, `speech_segments`, `speech_ms`, `silence_ms`.
- `interview_events` — every proctor signal with `event_type`, `severity`, `metadata`, `snapshot_path`, `client_ts`.
- `interview_reports` — performance, English proficiency and fluency, smartness, integrity, `end_reason`, generated summary.

RLS: admins read everything through `requireAdminUser()` from [lib/supabase/admin.ts](#). Anonymous student rows are written only by service-role routes after token and cookie validation.

Route map

Admin, authorised with the existing `requireAdminUser()` helper:

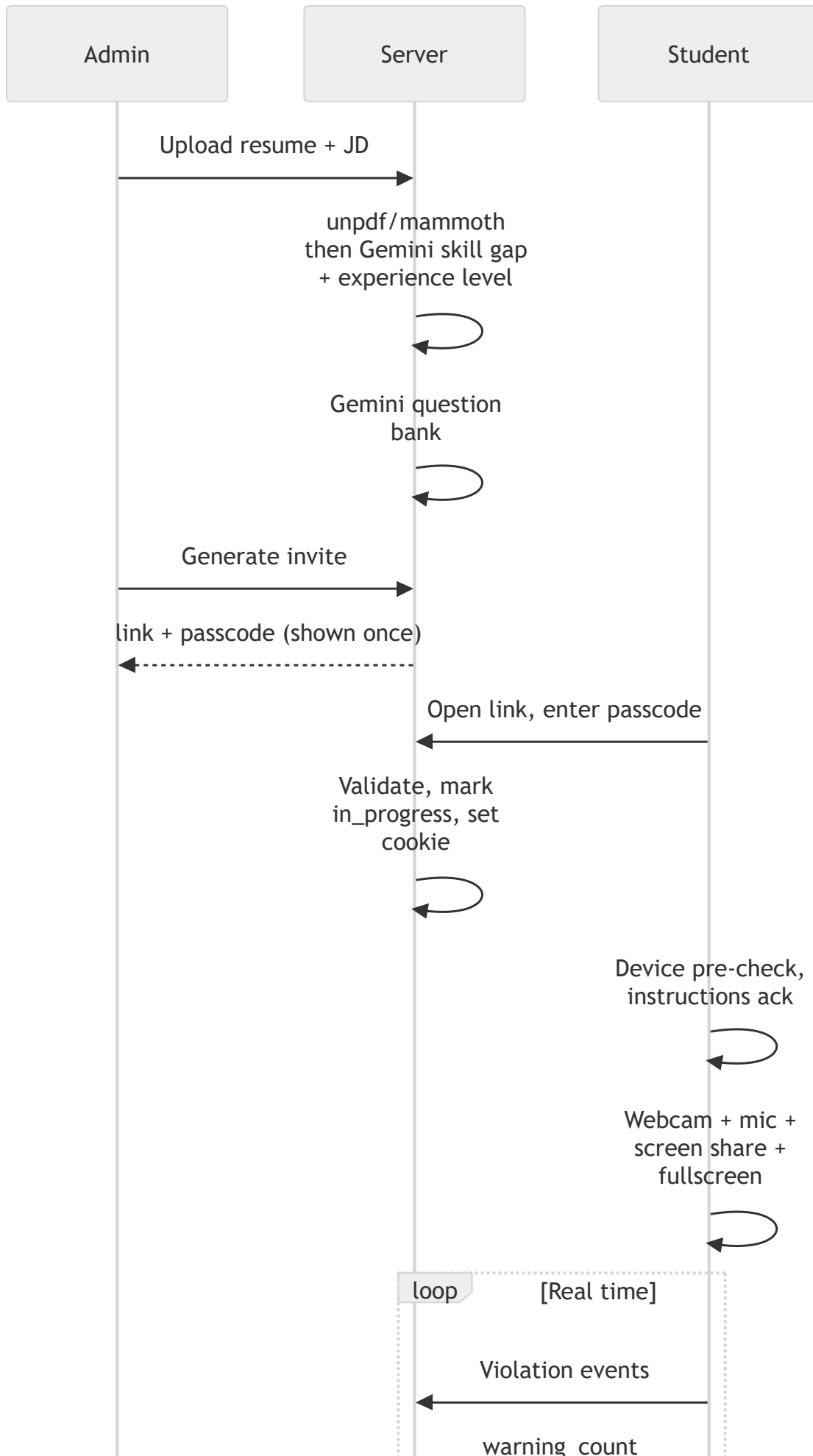
- `POST /api/interviews` — upload resume and JD
- `POST /api/interviews/[id]/parse` — extract text, Gemini structures skill gap and experience level
- `POST /api/interviews/[id]/generate` — Gemini interview question bank

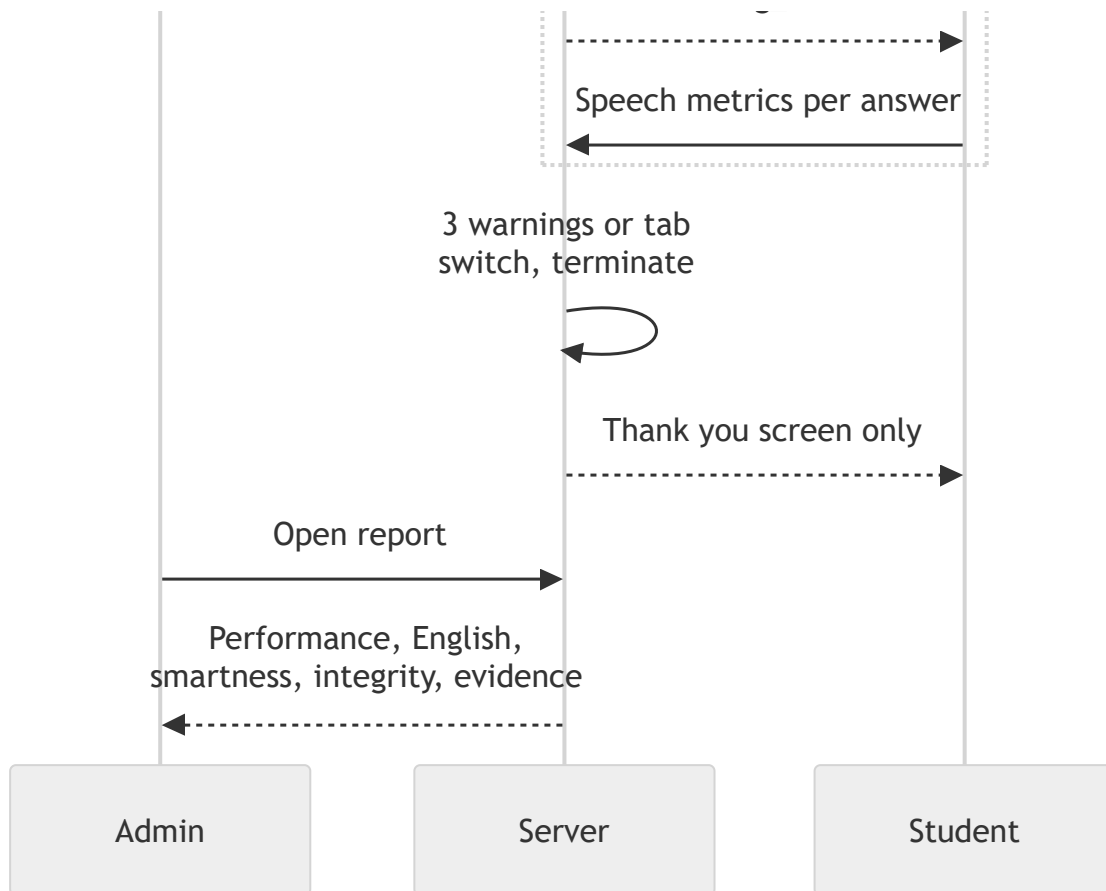
- `POST /api/interviews/[id]/invite` — returns link and passcode **once**
- `POST /api/interviews/[id]/invite/revoke` — kill a link before use
- `GET /api/admin/interviews/[id]/report` — the full analysis

Student, gated by token plus httpOnly cookie, never returns scores:

- `GET /assess/[token]` — passcode gate page
 - `POST /api/assess/[token]/verify` — script-compare the passcode, mark `in_progress`, set the cookie
 - `POST /api/assess/[token]/start` — after instructions acknowledged and media granted
 - `POST /api/assess/[token]/answer` — submits the answer transcript and its speech metrics
 - `POST /api/assess/[token]/events` — returns `{ warning_count, terminated }`
 - `POST /api/assess/[token]/finish` — returns only `{ message: "Your test is submitted. We will come back to you in sometime." }`
-

Full flow





Session integrity details that are easy to miss

- **Passcode delivery:** the link and passcode are shown to the admin once and are sent to the student out of band (email or message by the admin). The passcode is never embedded in the URL.
- **Link expiry:** invites carry a TTL, default 72 hours, and can be revoked by the admin at any time.
- **Pre-flight check:** before the instructions screen the app verifies browser support, camera, mic, and measures achievable inference FPS, warning the student if their device is too slow.
- **Refresh and network drop:** the bound session cookie lets the same browser resume the same session. A different browser or a cleared cookie is rejected as already used. Answers are persisted per question so a drop loses at most the current answer.
- **Timers:** per-question and total session limits, both configurable; expiry auto-submits rather than discarding work.
- **Student-visible copy is locked:** no scores, no correct answers, no integrity percentage, at any point.

Report contents for the admin

One page per session:

- Candidate summary, JD summary, and the skill gap that drove the questions
- Performance score with per-question breakdown and Gemini feedback
- English: overall score, CEFR band, grammar, vocabulary, coherence, plus fluency metrics (WPM, pause ratio, filler rate)

- Smartness composite with its sub-scores
- Integrity score, `end_reason`, and a chronological violation timeline with counts and snapshots
- Full transcripts per question

Delivery is the dashboard page by default. Stage 8 includes an optional email notification, which needs a provider such as Resend to be added since none is installed today.

Build order

- Stage 1 Foundation: migration, private `interview-docs` bucket, Zod types, invite token and passcode helpers with unit tests.
- Stage 2 Admin intake: upload route, `unpdf` and `mammoth` extractors, Gemini parse, question generation keyed to experience versus JD seniority, admin UI to create a session and copy the link and passcode.
- Stage 3 Student gate: passcode page, one-time consume, session cookie binding, device pre-check, instructions screen listing every rule and the 3-warning policy.
- Stage 4 Media locks: extend `lib/quizzes/proctor.ts` into `lib/proctor/`, add mic, fullscreen lock, tab-switch terminate, permission-revoke grace.
- Stage 5 Vision worker: MediaPipe worker with the three models, debounced rule engine, event POST, warning banner, auto-terminate, evidence snapshots.
- Stage 6 Speech and English: Silero VAD noise detection, Web Speech transcripts, real-time fluency metrics, long-silence handling, spoken answer turns.
- Stage 7 Scoring: Gemini rubrics for interview performance, spoken English proficiency, smartness composite, integrity derivation.
- Stage 8 Report and hardening: admin report page, thank-you copy lock, optional email, retention job, tests.

Verification per stage: `npm run test` and `npm run lint`, plus a manual check that the student `finish` response body contains no scores.

Risks

- Three vision models at once is the main perf risk on low-end laptops. Mitigation: GPU delegate, worker with OffscreenCanvas, staggered inference rates, and the pre-flight FPS check.
- Object detection false positives, for example a mug read as a phone. Mitigation: 0.5 confidence floor plus 3 consecutive frames, and a snapshot so the admin can verify.
- Web Speech API is Chromium-only, and English scoring depends on it. Mitigation: browser check before start, Gemini audio transcription as the fallback path.
- Fluency metrics degrade with a poor microphone. Mitigation: the pre-flight check measures input level, and the report flags low audio quality so scores are read in context.
- Immediate termination on tab switch will catch accidental alt-tabs. The config flag lets you soften it after a pilot.